

ГЛАВА 11

АССОЦИАТИВНЫЕ КОНТЕЙНЕРЫ

В этой главе...

11.1. Использование ассоциативных контейнеров	538
11.2. Обзор ассоциативных контейнеров	541
11.3. Работа с ассоциативными контейнерами	548
11.4. Неупорядоченные контейнеры	566
Резюме	570
Термины	570

Ассоциативные контейнеры кардинально отличаются от последовательных: элементы в ассоциативном контейнере хранятся и предоставляются по ключу. Элементы последовательного контейнера, напротив, хранятся и предоставляются последовательно, по их позиции в контейнере.

Хотя поведение ассоциативных контейнеров по большей части одинаково, у последовательных контейнеров оно отличается и зависит от способа использования ключей.

Ассоциативные контейнеры (associative container) обеспечивают быстрый поиск и предоставление элементов по ключу. Двумя первичными типами ассоциативных контейнеров являются `map` (карта) и `set` (набор). Элементами контейнера `map` являются *пары ключ-значение* (key-value pair): ключ выступает в роли индекса, а значение представляет собой хранимые в контейнере данные. Контейнер `set` содержит только ключи и предоставляет эффективные способы запроса на проверку наличия определенного ключа. Набор можно было бы использовать для хранения слов, которые следует проигнорировать при некой обработке текста. Карту можно использовать для словаря: слово было бы ключом, а его определение — значением.

Библиотека предоставляет восемь ассоциативных контейнеров (табл. 11.1), которые различаются по трем факторам: (1) они являются набором (`set`) или картой `map`; (2) они требуют уникальных ключей или допускает их совпадение; (3) они хранят элементы упорядочено или нет. В именах контейнеров, допускающих совпадение ключей, присутствует слово `multi`; имена контейнеров, не

упорядочивающих хранимые ключи начинаются со слова `unordered`. Следовательно, `unordered_multi_set` — это набор, не требующий уникальных ключей и хранящий элементы неупорядоченными, в то время как `set` — это набор с уникальными ключами, которые хранятся упорядочено. Для организации своих элементов неупорядоченные контейнеры используют хеш-функцию. Подробно хеш-функции рассматриваются в разделе 11.4 (стр. 567).

Таблица 11.1. Типы ассоциативных контейнеров

Элементы упорядочиваются по ключу	
<code>map</code>	Ассоциативный массив, хранящий пары ключ–значение
<code>set</code>	Контейнер, в котором ключ является значением
<code>multimap</code>	Карта, допускающая совпадение ключей
<code>multiset</code>	Набор, допускающий совпадение ключей
Неупорядоченные коллекции	
<code>unordered_map</code>	Карта, организованная по хеш-функции
<code>unordered_set</code>	Набор, организованный по хеш-функции
<code>unordered_multimap</code>	Хешированная карта; ключи могут повторяться
<code>unordered_multiset</code>	Хешированный набор; ключи могут повторяться

Типы `map` и `multimap` определены в заголовке `map`; классы `set` и `multiset` — в заголовке `set`; неупорядоченные версии контейнеров определены в заголовках `unordered_map` и `unordered_set` соответственно.



11.1. Использование ассоциативных контейнеров

Хотя большинство программистов знакомы с такими структурами данных, как векторы и списки, многие из них никогда не используют ассоциативные структуры данных. Прежде чем перейти к подробностям того, как библиотека поддерживает эти типы, имеет смысл начать с примеров использования этих контейнеров.

Карта (тип `map`) — это коллекция пар ключ–значение. Например, каждая пара может содержать имя человека как ключ и номер его телефона как значение. О такой структуре данных говорят, что она “сопоставляет имена с номерами телефонов”. Тип `map` зачастую называют *ассоциативным массивом* (*associative array*). Ассоциативный массив похож на обычный массив, но его индексы не обязаны быть целыми числами. Значения в карте находят по ключу

чу, а не по их позиции. В карте имен и номеров телефонов имя человека использовалось бы как индекс для поиска номера телефона этого человека.

В отличие от карты, набор — это просто коллекция ключей. Набор полезен тогда, когда необходимо знать, имеется ли в нем значение. Например, банк мог бы определить набор `bad_checks` для хранения имен личностей, которые выписывали фальшивые чеки. Прежде чем принять чек, этот банк запросил бы набор `bad_checks` и убедился, есть ли в нем имя клиента.

Использование контейнера `map`

Классическим примером применения ассоциативного массива является программа подсчета слов:

```
// подсчитать, сколько раз каждое слово встречается во вводе
map<string, size_t> word_count; // пустая карта строк и чисел
string word;
while (cin >> word)
    ++word_count[word]; // получить и прирастить счетчик слов
for (const auto &w : word_count) // для каждого элемента карты
    // отобразить результаты
    cout << w.first << " occurs " << w.second
        << ((w.second > 1) ? " times" : " time") << endl;
```

Эта программа читает ввод и сообщает, сколько раз встречается каждое слово.

Подобно последовательным контейнерам, ассоциативные контейнеры являются шаблонами (см. раздел 3.3, стр. 141). Чтобы определить карту, следует указать типы ключа и значения. В этой программе карта хранит элементы, ключи которых имеют тип `string`, а значения — тип `size_t` (см. раздел 3.5.2, стр. 165). При индексации карты `word_count` строка используется как индекс, а возвращаемый счетчик типа `size_t` связан с этой строкой.

Цикл `while` читает слова со стандартного устройства ввода по одному за раз. Он использует каждое слово для индексирования карты `word_count`. Если слова еще нет в карте, оператор индексирования создает новый элемент, ключом которого будет слово, а значением 0. Независимо от того, должен ли быть создан элемент, его значение увеличивается.

Как только весь ввод прочитан, серийный оператор `for` (см. раздел 3.2.3, стр. 138) перебирает карту выводя каждое слово и соответствующий счетчик. При получении элемента из карты возвращается объект типа `pair` (пара), рассматриваемого в разделе 11.2.3 (стр. 545). Если не вдаваться в подробности, то `pair` — это шаблон типа, который содержит две открытые переменные-члена по имени `first` (первый) и `second` (второй). У используемых картой пар член `first` является ключом, а `second` — соответствующим значением. Таким образом, оператор вывода должен отобразить каждое слово и связанный с ним счетчик.

Если бы эта программа была запущена для текста первого параграфа данного раздела, то вывод был бы таким:

```
Although occurs 1 time
Before occurs 1 time
an occurs 1 time
and occurs 1 time
...
```

Использование контейнера `set`

Логичным усовершенствованием создаваемой программы будет игнорирование таких распространенных слов, как “the”, “and”, “or” и т.д. Для хранения игнорируемых слов будет использован набор, а подсчитываться будут только те слова, которые отсутствуют в этом наборе:

```
// подсчитать, сколько раз каждое слово встречается во вводе
map<string, size_t> word_count; // пустая карта строк и чисел
set<string> exclude = {"The", "But", "And", "Or", "An", "A",
                      "the", "but", "and", "or", "an", "a"};

string word;
while (cin >> word)
    // подсчитать только не исключенные слова
    if (exclude.find(word) == exclude.end())
        ++word_count[word]; // получить и прирастить счетчик слов
```

Подобно другим контейнерам, `set` является шаблоном. Чтобы определить набор, следует указать тип его элементов, которым в данном случае будет `string`. Подобно последовательным контейнерам, для элементов ассоциативного контейнера применима списочная инициализация (см. раздел 3.3.6, стр. 432). Набор `exclude` будет содержать 12 слов, которые следует игнорировать.

Важнейшее различие между этой программой и предыдущей в том, что перед подсчетом каждого слова оно проверяется на принадлежность к набору исключенных. Для этого используется оператор `if`:

```
// подсчитать только не исключенные слова
if (exclude.find(word) == exclude.end())
```

Вызов функции `find()` возвращает итератор. Если заданный ключ находится в наборе, итератор указывает на него. Если элемент не найден, функция `find()` возвращает итератор на элемент после конца. В этой версии счетчик слов изменяется, только если слово не находится в наборе `exclude`.

Если запустить эту версию для того же ввода, что и прежде, вывод будет таким:

```
Although occurs 1 time
Before occurs 1 time
are occurs 1 time
as occurs 1 time
```

Упражнения раздела 11.1

Упражнение 11.1. Опишите различия между картой и вектором.

Упражнение 11.2. Приведите пример того, когда наиболее полезен контейнер `list`, `vector`, `deque`, `map` и `set`.

Упражнение 11.3. Напишите собственную версию программы подсчета слов.

Упражнение 11.4. Усовершенствуйте свою программу так, чтобы игнорировать регистр и пунктуацию. Т.е. слова “example” и “Example”, например, должны увеличить тот же счетчик.

11.2. Обзор ассоциативных контейнеров

Ассоциативные контейнеры (и упорядоченные, и неупорядоченные) поддерживают общие функции контейнеров, описанные в разделе 9.2 (стр. 423) и перечисленные в табл. 9.2 (стр. 423). Ассоциативные контейнеры *не поддерживают* функции, специфические для последовательных контейнеров, такие как `push_front()` или `back()`. Поскольку элементы хранятся на основании их ключа, эти операции были бы бессмысленны для ассоциативных контейнеров. Кроме того, ассоциативные контейнеры не поддерживают конструкторы и функции вставки, получающие значение элемента и его позицию.

Кроме функций, общих для всех контейнеров, ассоциативные контейнеры предоставляют некоторые функции (табл. 11.7, стр. 558) и псевдонимы типов (табл. 11.3, стр. 548), которых нет у последовательных контейнеров. Помимо этого, неупорядоченные контейнеры предоставляют функции настройки производительности их хеша, которые рассматриваются в разделе 11.4 (стр. 567).

Итераторы ассоциативных контейнеров двунаправленны (см. раздел 10.5.1, стр. 525).



11.2.1. Определение ассоциативного контейнера



Как только что упоминалось, при определении карты следует указать типы ключа и значения; при определении набора задают только тип ключа, поскольку значения у него нет. Каждый из ассоциативных контейнеров имеет стандартный конструктор, который создает пустой контейнер заданного типа. Ассоциативный контейнер можно также инициализировать копией другого контейнера того же типа или диапазоном значений, тип которых может быть приведен к типу контейнера. По новому стандарту возможна также списочная инициализация элементов:

```
map<string, size_t> word_count; // пустая карта
// списочная инициализация
```

```
set<string> exclude = {"the", "but", "and", "or", "an", "a",
                      "The", "But", "And", "Or", "An", "A"};
// три элемента; authors сопоставляет фамилию с именем
map<string, string> authors = { {"Joyce", "James"},
                               {"Austen", "Jane"},
                               {"Dickens", "Charles"} };
```

Как обычно, тип инициализаторов должен быть преобразуемым в тип контейнера. Типом элемента набора является тип ключа.

При инициализации карты следует предоставить и ключ, и значение. Каждая пара ключ–значение заключается в фигурные скобки, {*ключ*, *значение*}, означая, что вместе элементы формируют единый элемент карты. Первый элемент каждой пары — это ключ, второй — значение. Таким образом, карта `authors` сопоставляет фамилии с именами и инициализируется тремя элементами.

Инициализация контейнеров `multimap` и `multiset`

Ключи в контейнерах `map` и `set` должны быть уникальными; с каждым ключом может быть сопоставлен только один элемент. У контейнеров `multimap` и `multiset` такого ограничения нет; вполне допустимо несколько элементов с тем же ключом. Например, у использованной для подсчета слов карты должен быть только один элемент, содержащий некое слово. С другой стороны, у словаря может быть несколько определений того же слова.

Следующий пример иллюстрирует различия между контейнерами с уникальными ключами и таковыми с не уникальными ключами. Сначала необходимо создать вектор целых чисел `ivec` на 20 элементов: две копии каждого из целых чисел от 0 до 9 включительно. Этот вектор будет использован для инициализации контейнеров `set` и `multiset`:

```
// определить вектор из 20 элементов, содержащий две копии каждого
// числа от 0 до 9
vector<int> ivec;
for (vector<int>::size_type i = 0; i != 10; ++i) {
    ivec.push_back(i);
    ivec.push_back(i); // сдублировать каждое число
}
// iset содержит уникальные элементы ivec;
// miset содержит все 20 элементов
set<int> iset(ivec.cbegin(), ivec.cend());
multiset<int> miset(ivec.cbegin(), ivec.cend());
cout << ivec.size() << endl; // выводит 20
cout << iset.size() << endl; // выводит 10
cout << miset.size() << endl; // выводит 20
```

Хотя набор `iset` был инициализирован значениями всего контейнера `ivec`, он содержит только десять элементов: по одному для каждого уникального элемента вектора `ivec`. С другой стороны, контейнер `miset` содержит 20 элементов, сколько и вектор `ivec`.

Упражнения раздела 11.2.1

Упражнение 11.5. Объясните различие между картой и набором. Когда имеет смысл использовать один, а когда другой?

Упражнение 11.6. Объясните различия между набором и списком. Когда имеет смысл использовать один, а когда другой?

Упражнение 11.7. Определите карту, ключ которой является фамилией семьи, а значение — вектором имен детей. Напишите код, способный добавлять новые семьи и новых детей в существующие семьи.

Упражнение 11.8. Напишите программу, которая хранит исключенные слова в векторе, а не в наборе. Каковы преимущества использования набора?



11.2.2. Требования к типу ключа

Ассоциативные контейнеры налагают ограничения на тип ключа. Требования для ключей неупорядоченных контейнеров рассматриваются в разделе 11.4 (стр. 568). У упорядоченных контейнеров (`map`, `multimap`, `set` и `multiset`) тип ключа должен определять способ сравнения элементов. По умолчанию для сравнения ключей библиотека использует оператор `<` типа ключа. В наборах тип ключа соответствует типу элемента; в картах тип ключа — тип первого элемента пары. Таким образом, типом ключа карты `word_count` (см. раздел 11.1, стр. 539) будет `string`. Аналогично типом ключа набора `exclude` также будет `string`.



Вызываемые объекты, переданные алгоритму сортировки (см. раздел 10.3.1, стр. 494), должны соответствовать тем же требованиям, что и ключи в ассоциативном контейнере.

Типы ключей упорядоченных контейнеров

Подобно тому, как собственный оператор сравнения можно предоставить алгоритму (см. раздел 10.3, стр. 494), собственный оператор можно также предоставить для использования вместо оператора `<` ключей. Заданный оператор должен обеспечить *строгое сравнение* (*strict weak ordering*) для типа ключа. Строгое сравнение можно считать оператором “меньше”, хотя наша функция могла бы использовать более сложную процедуру. Однако самостоятельно определяемая функция сравнения должна обладать свойствами, описанными ниже.

- Два ключа не могут быть “меньше” друг друга; если ключ `k1` “меньше”, чем `k2`, то `k2` никогда не должен быть “меньше”, чем `k1`.
- Если ключ `k1` “меньше”, чем `k2`, и ключ `k2` “меньше”, чем `k3`, то ключ `k1` должен быть “меньше”, чем `k3`.

- Если есть два ключа и ни один из них не “меньше” другого, то эти ключи “эквивалентны”. Если ключ `k1` “эквивалентен” ключу `k2` и ключ `k2` “эквивалентен” ключу `k3`, то ключ `k1` должен быть “эквивалентен” ключу `k3`.

Если два ключа эквивалентны (т.е. если ни один не “меньше” другого), то контейнер рассматривает их как равные. С этими ключами в карте будет ассоциирован только один элемент, и любой из них предоставит доступ к тому же значению.



На практике очень важно, чтобы тип, определяющий “обычный” оператор `<`, был применим в качестве ключа.

Использование функции сравнения для типа ключа

Тип оператора, используемого контейнером для организации своих элементов, является частью типа этого контейнера. Чтобы определить собственный оператор, следует предоставить тип этого оператора при определении типа ассоциативного контейнера. Тип оператора указывают после типа элемента в угловых скобках, используемых для указания типа определяемого контейнера.

Каждый тип в угловых скобках — это только тип. Специальный оператор сравнения (тип которого должен совпадать с типом, указанным в угловых скобках) предоставляется как аргумент конструктора при создании контейнера.

Например, невозможно непосредственно определить контейнер `multiset` объектов класса `Sales_data`, поскольку класс `Sales_data` не имеет оператора `<`. Но для этого можно использовать функцию `compareIsbn()` из упражнений раздела 10.3.1 (стр. 496). Эта функция обеспечивает строгое сравнение на основании ISBN двух объектов класса `Sales_data`. Функция `compareIsbn()` должна выглядеть примерно так:

```
bool compareIsbn(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() < rhs.isbn();
}
```

Чтобы использовать собственный оператор, следует определить контейнер `multiset` с двумя типами: типом ключа `Sales_data` и типом сравнения, являющимся типом указателя на функцию (см. раздел 6.7, стр. 322), способным указывать на функцию `compareIsbn()`. Когда определяют объекты этого типа, предоставляют указатель на функцию, которую предстоит использовать. В данном случае предоставляется указатель на функцию `compareIsbn()`:

```
// в программе может быть несколько транзакций с тем же ISBN
// элементы bookstore упорядочены по ISBN
multiset<Sales_data, decltype(compareIsbn)*>
    bookstore(compareIsbn);
```


Здесь для определения типа оператора используется спецификатор `decltype`. При использовании спецификатора `decltype` для получения указателя на функцию следует добавить символ `*` для обозначения использования указателя на заданный тип функции (см. раздел 6.7, стр. 326). Инициализацию `bookstore` осуществляет функция `compareIsbn()`. Это означает, что при добавлении элементов в `bookstore` они будут упорядочены при вызове функции `compareIsbn()`. Таким образом, элементы `bookstore` будут упорядочены по их члену ISBN. Аргумент конструктора можно записать как `compareIsbn()`, вместо `&compareIsbn()`, поскольку при использовании имени функции оно автоматически преобразуется в указатель, если это нужно (см. раздел 6.7, стр. 323). С тем же результатом можно написать `&compareIsbn()`.

Упражнения раздела 11.2.2

Упражнение 11.9. Определите карту, которая ассоциирует слова со списком номеров строк, в которых оно встречается.

Упражнение 11.10. Можно ли определить карту для типов `vector<int>::iterator` и `int`? А для типов `list<int>::iterator` и `int`? Если нет, то почему?

Упражнение 11.11. Переопределите `bookstore`, не используя спецификатор `decltype`.

11.2.3. Тип `pair`

Прежде чем перейти к рассмотрению действий с ассоциативными контейнерами, имеет смысл ознакомиться с библиотечным типом `pair` (пара), определенным в заголовке `utility`.

Объект типа `pair` хранит две переменные-члена. Подобно контейнерам, тип `pair` является шаблоном, позволяющим создавать конкретные типы. При создании пары следует предоставить имена двух типов, которые будут типами ее двух переменных-членов. Совпадать эти типы вовсе не обязаны.

```
pair<string, string> anon;           // содержит две строки
pair<string, size_t> word_count;    // содержит строку и целое число
pair<string, vector<int>> line;      // содержит строку и vector<int>
```

При создании объекта пары без указания инициализирующих значений используются стандартные конструкторы типов его переменных-членов. Таким образом, пара `anon` содержит две пустые строки, а пара `line` — пустую строку и пустой вектор целых чисел. Значением переменной-члена типа `int` в паре `word_count` будет 0, а его переменная-член типа `string` окажется инициализирована пустой строкой.

Можно также предоставить инициализаторы для каждого члена пары:

```
pair<string, string> author("James", "Joyce");
```

Этот код создает пару по имени `author`, инициализированную значениями "James" и "Joyce".

В отличие от других библиотечных типов, переменные-члены класса `pair` являются открытыми (см. раздел 7.2, стр. 349). Эти члены — `first` (первый) и `second` (второй) соответственно. К ним можно обращаться непосредственно, используя обычный точечный оператор (см. раздел 1.5.2 стр. 51), как, например, было сделано в операторе вывода программы подсчета слов на стр. 539:

```
// отобразить результаты
cout << w.first << " occurs " << w.second
    << ((w.second > 1) ? " times" : " time") << endl;
```

где `w` — ссылка на элемент карты. Элементами карты являются пары. В данном операторе выводится переменная-член `first` элемента, являющаяся ключом, затем переменная-член `second` элемента, являющаяся счетчиком. Библиотека определяет весьма ограниченный набор операций с парами, который приведен в табл. 11.2.

Таблица 11.2. Операции с парами

<code>pair<T1, T2> p;</code>	<code>p</code> — пара с переменными-членами типов <code>T1</code> и <code>T2</code> , инициализированными значением по умолчанию (см. раздел 3.3.1, стр. 143)
<code>pair<T1, T2> p(v1, v2);</code>	<code>p</code> — пара с переменными-членами типов <code>T1</code> и <code>T2</code> , инициализированными значениями <code>v1</code> и <code>v2</code> соответственно
<code>pair<T1, T2> p = {v1, v2};</code>	Эквивалент <code>p(v1, v2)</code>
<code>make_pair(v1, v2)</code>	Возвращает пару, инициализированную значениями <code>v1</code> и <code>v2</code> . Тип пары выводится из типов значений <code>v1</code> и <code>v2</code>
<code>p.first</code>	Возвращает открытую переменную-член <code>first</code> пары <code>p</code>
<code>p.second</code>	Возвращает открытую переменную-член <code>second</code> пары <code>p</code>
<code>p1 < op < p2</code>	Операторы сравнения (<code><</code> , <code>></code> , <code><=</code> , <code>>=</code>). Сравнение осуществляется подобно упорядочиванию в словаре, т.е. оператор <code><</code> возвращает значение <code>true</code> в случае, если <code>p1.first < p2.first</code> или <code>!(p2.first < p1.first) && p1.second < p2.second</code>
<code>p1 == p2, p1 != p2</code>	Две пары равны, если их первый и второй члены соответственно равны. При сравнении используется оператор <code>==</code> хранимых элементов

Функция для создания объектов типа `pair`

**C++
11** Предположим, некая функция должна вернуть значение типа `pair`. По новому стандарту возможна списочная инициализация возвращаемого значения (см. раздел 6.3.2, стр. 296):

```
pair<string, int>
process(vector<string> &v)
{
    // обработка v
    if (!v.empty())
        return {v.back(), v.back().size()}; // списочная инициализация
    else
        return pair<string, int>(); // возвращаемое значение создано явно
}
```

Если вектор `v` не пуст, возвращается пара, состоящая из последней строки в векторе `v` и размера этой строки. В противном случае явно создается и возвращается пустая пара.

В прежних версиях языка C++ нельзя было использовать инициализаторы в скобках для возвращения типа, подобного `pair`. Вместо этого можно было написать оба оператора `return` как явно созданное возвращаемое значение:

```
if (!v.empty())
    return pair<string, int>(v.back(), v.back().size());
```

В качестве альтернативы можно использовать функцию `make_pair()` для создания новой пары соответствующего типа из двух аргументов:

```
if (!v.empty())
    return make_pair(v.back(), v.back().size());
```

Упражнения раздела 11.2.3

Упражнение 11.12. Напишите программу, читающую последовательность строк и целых чисел, сохраняя каждую прочитанную пару в объекте класса `pair`. Сохраните пары в векторе.

Упражнение 11.13. Существует по крайней мере три способа создания пар в программе предыдущего упражнения. Напишите три версии программы, создающей пары каждым из этих способов. Укажите, какая из форм проще и почему.

Упражнение 11.14. Дополните карту фамилий семей и их детей, написанную для упражнения в разделе 11.2.1 (стр. 543), вектором пар, содержащих имя ребенка и день его рождения.

11.3. Работа с ассоциативными контейнерами

В дополнение к типам, перечисленным в табл. 9.2 (стр. 423), ассоциативные контейнеры определяют типы, перечисленные в табл. 11.3. Они представляют типы ключа и значения контейнера.

Таблица 11.3. Псевдонимы дополнительных типов ассоциативных контейнеров

<code>key_type</code>	Тип ключа контейнера
<code>mapped_type</code>	Тип, ассоциированный с каждым ключом; <i>только для типа <code>map</code></i>
<code>value_type</code>	Для наборов то же, что и <code>key_type</code> . Для карт — <code>pair<const key_type, mapped_type></code>

Для контейнеров типа `set` типы `key_type` и `value_type` совпадают; содержащиеся в наборе данные являются ключами. Элементами карты являются пары ключ–значение. Таким образом, каждый ее элемент — объект класса `pair`, содержащий ключ и связанное с ним значение. Поскольку ключ элемента изменить нельзя, ключевая часть этих пар константна:

```
set<string>::value_type v1;      // v1 - string
set<string>::key_type v2;       // v2 - string
map<string, int>::value_type v3; // v3 - pair<const string, int>
map<string, int>::key_type v4;   // v4 - string
map<string, int>::mapped_type v5; // v5 - int
```

Подобно последовательным контейнерам (см. раздел 9.2.2, стр. 429), для доступа к члену класса, например типа `map<string, int>::key_type`, используется оператор области видимости.

Тип `mapped_type` определен только для типов карт (`unordered_map`, `unordered_multimap`, `multimap` и `map`).

11.3.1. Итераторы ассоциативных контейнеров

При обращении к значению итератора возвращается ссылка на значение типа `value_type` контейнера. В случае карты типом `value_type` является пара, переменная-член `first` которой содержит константный ключ, а переменная-член `second` — значение:

```
// получить итератор на элемент контейнера word_count
auto map_it = word_count.begin();
// *map_it - ссылка на объект типа pair<const string, size_t>
cout << map_it->first;      // отобразить ключ элемента
cout << " " << map_it->second; // отобразить значение элемента
map_it->first = "new key";   // ошибка: ключ является константой
++map_it->second;           // ok: значение можно изменить, используя итератор
```



Не следует забывать, что типом `value_type` карты является `pair` и что можно изменять ее значение, но не ключ.

Итераторы наборов константны

Хотя типы наборов определяют типы `iterator` и `const_iterator`, оба типа итераторов предоставляют доступ к элементам в наборе только для чтения. Подобно тому, как нельзя изменить ключевую часть элемента карты, ключи в наборе также константны. Итератор набора можно использовать только для чтения, но не для записи значения элемента:

```
set<int> iset = {0,1,2,3,4,5,6,7,8,9};
set<int>::iterator set_it = iset.begin();
if (set_it != iset.end()) {
    *set_it = 42;           // ошибка: ключи набора только для чтения
    cout << *set_it << endl; // ок: позволяет читать ключ
}
```

Перебор ассоциативного контейнера

Типы `map` и `set` поддерживают все функции `begin()` и `end()` из табл. 9.2 (стр. 423). Как обычно, эти функции можно использовать для получения итераторов, позволяющих перебрать контейнер. Например, цикл вывода результатов программы подсчета слов со стр. 539 можно переписать следующим образом:

```
// получить итератор на первый элемент
auto map_it = word_count.cbegin();
// сравнить текущий итератор с итератором после конца
while (map_it != word_count.cend()) {
    // обратиться к значению итератора, чтобы отобразить
    // пару ключ-значение элемента
    cout << map_it->first << " occurs "
         << map_it->second << " times" << endl;
    ++map_it; // прирастить итератор, чтобы перейти на следующий элемент
}
```

Условие цикла `while` и инкремент итератора в теле цикла такие же как в программах вывода содержимого векторов или строк. Итератор `map_it` инициализирован позицией первого элемента контейнера `word_count`. Пока итератор не равен значению, возвращенному функцией `end()`, возвращается текущий элемент, а затем происходит приращение итератора. Оператор вывода обращается к значению итератора `map_it` для получения членов пары, оставаясь в остальном тем же, что и в первоначальной программе.



Вывод этой программы имеет алфавитный порядок. При использовании итераторов для перебора контейнеров `map`, `multimap`, `set` и `multiset` они возвращают элементы в порядке возрастания ключа.

Ассоциативные контейнеры и алгоритмы

Как правило, с ассоциативными контейнерами обобщенные алгоритмы (см. главу 10) не используются. Тот факт, что ключи константны, означает невозможность передачи итераторов ассоциативных контейнеров алгоритмам, которые пишут или переупорядочивают элементы контейнеров. Таким алгоритмам нужна возможность записи в элементы. Элементы всех типов наборов константны, а у всех типов карт константным является первый член пары.

Ассоциативные контейнеры применимы с теми алгоритмами, которые только читают элементы. Однако большинство этих алгоритмов осуществляет поиск в последовательности. Поскольку поиск элементов в ассоциативном контейнере осуществляется быстро (по ключу), как правило, не имеет смысла использовать для них обобщенный алгоритм поиска. Например, как будет продемонстрировано в разделе 11.3.5 (стр. 558), ассоциативные контейнеры определяют функцию-член `find()`, позволяющую непосредственно выбрать элемент с заданным ключом. Для поиска элемента можно использовать обобщенный алгоритм `find()`, но он осуществляет последовательный поиск. Поэтому намного быстрее использовать функцию-член `find()` класса контейнера, чем вызывать обобщенную версию.

На практике, если это вообще происходит, ассоциативный контейнер используется с алгоритмами в качестве исходной последовательности или последовательности назначения. Например, обобщенный алгоритм `copy()` можно использовать для копирования элементов ассоциативного контейнера в другую последовательность. Точно так же адаптер `inserter` можно использовать для связи итератора вставки (см. раздел 10.4.1, стр. 514) с ассоциативным контейнером. Адаптер `inserter` позволяет использовать ассоциативный контейнер как место назначения для другого алгоритма.

Упражнения раздела 11.3.1

Упражнение 11.15. Каковы типы `mapped_type`, `key_type` и `value_type` карты, переменные-члены пар которой имеют типы `int` и `vector<int>`?

Упражнение 11.16. Используя итератор карты, напишите выражение, присваивающее значение элементу.

Упражнение 11.17. С учетом того, что `c` — контейнер `multiset` строк, а `v` — вектор строк, объясните следующие вызовы. Укажите, допустим ли каждый из них:

```
copy(v.begin(), v.end(), inserter(c, c.end()));
copy(v.begin(), v.end(), back_inserter(c));
copy(c.begin(), c.end(), inserter(v, v.end()));
copy(c.begin(), c.end(), back_inserter(v));
```

Упражнение 11.18. Перепишите определение типа `map_it` из цикла на стр. 549, не используя ключевое слово `auto` или `decltype`.

Упражнение 11.19. Определите переменную, инициализированную вызовом функции `begin()` контейнера `multiset` по имени `bookstore` из раздела 11.2.2 (стр. 543). Определите тип переменной, не используя ключевое слово `auto` или `decltype`.

11.3.2. Добавление элементов

Функция-член `insert()` (табл. 11.4) добавляет один элемент или диапазон элементов в контейнер. Поскольку карта и набор (и их неупорядоченные версии) содержат уникальные ключи, попытка вставки уже присутствующего элемента не имеет никакого эффекта:

```
vector<int> ivec = {2,4,6,8,2,4,6,8};    // ivec содержит восемь элементов
set<int> set2;                          // пустой набор
set2.insert(ivec.cbegin(), ivec.cend()); // set2 имеет четыре элемента
set2.insert({1,3,5,7,1,3,5,7});         // теперь set2 имеет восемь элементов
```

Таблица 11.4. Функция `insert()` ассоциативного контейнера

<code>c.insert(v)</code> <code>c.emplace(args)</code>	<code>v</code> — объект типа <code>value_type</code> ; аргументы <code>args</code> используются при создании элемента. Элементы карты и набора вставляются (или создаются), только если элемента с данным ключом еще нет в контейнере <code>c</code> . Возвращает пару, содержащую итератор на элемент с заданным ключом и логическое значение, указывающее, был ли вставлен элемент. У контейнеров <code>multimap</code> и <code>multiset</code> осуществляется вставка (или создание) заданного элемента и возвращение итератора на новый элемент
<code>c.insert(b, e)</code> <code>c.insert(il)</code>	Итераторы <code>b</code> и <code>e</code> обозначают диапазон значений типа <code>c::value_type</code> ; <code>il</code> — заключенный в скобки список таких значений. Возвращает <code>void</code> . У карты и набора вставляются элементы с ключами, которых еще нет в контейнере <code>c</code> . У контейнеров <code>multimap</code> и <code>multiset</code> вставляются все элементы диапазона
<code>c.insert(p, v)</code> <code>c.emplace(p, args)</code>	Подобны функциям <code>insert(v)</code> и <code>emplace(args)</code> , но используют итератор <code>p</code> как подсказку для начала поиска места хранения нового элемента. Возвращает итератор на элемент с заданным ключом

Версии функции `insert()`, получающие пару итераторов или список инициализации, работают подобно соответствующим конструкторам (см. раздел 11.2.1, стр. 541), но добавляется только первый элемент с заданным ключом.

Добавление элементов в карту

При вставке в карту следует помнить, что типом элемента является `pair`. Зачастую объекта `pair`, подлежащего вставке, нет. В этом случае пара создается в списке аргументов функции `insert()`:

```
// четыре способа добавления слова в word_count
word_count.insert({word, 1});
word_count.insert(make_pair(word, 1));
word_count.insert(pair<string, size_t>(word, 1));
word_count.insert(map<string, size_t>::value_type(word, 1));
```

**C++
11** Как уже упоминалось, по новому стандарту простейшим способом создания пары является инициализация списком аргументов в фигурных скобках. В качестве альтернативы можно вызвать функцию `make_pair()` или явно создать пару. Вот аргументы последнего вызова функции `insert()`:

```
map<string, size_t>::value_type(s, 1)
```

Он создает новый объект пары соответствующего типа для вставки в карту.

Проверка значения, возвращаемого функцией `insert()`

Значение, возвращенное функцией `insert()` (или `emplace()`), зависит от типа контейнера и параметров. Для контейнеров с уникальными ключами есть версии функций `insert()` и `emplace()`, которые добавляют один элемент и возвращают пару, сообщающую об успехе вставки. Первая переменная-член пары — итератор на элемент с заданным ключом; второй — логическое значение, указывающее на успех вставки элемента. Если такой ключ уже был в контейнере, то функция `insert()` не делает ничего, а логическая часть возвращаемого значения содержит `false`. Если такой ключ отсутствовал, то логическая часть содержит значение `true`.

Для примера перепишем программу подсчета слов с использованием функции `insert()`:

```
// более корректный способ подсчета слов во вводе
map<string, size_t> word_count; // пустая карта строк и чисел
string word;
while (cin >> word) {
    // вставляет элемент с ключом, равным слову, и значением 1;
    // если слово уже есть в word_count, insert() не делает ничего
    auto ret = word_count.insert({word, 1});
    if (!ret.second) // слово уже было в word_count
        ++ret.first->second; // приращение счетчика
}
```

Для каждой строки `word` осуществляется попытка вставки со значением 1. Если слово уже находится в карте, ничего не происходит. В частности, связанный со словом счетчик остается неизменным. Если слова еще нет в карте, оно добавляется, а значение его счетчика устанавливается в 1.

Оператор `if` проверяет логическую часть возвращаемого значения. Если это значение `false`, то вставка не произошла. Следовательно, слово уже было в карте `word_count`, поэтому следует увеличить значение связанного с ним счетчика.

Еще раз о синтаксисе

Оператор приращения счетчика в этой версии программы подсчета слов трудно понять. Разобрать это выражение будет существенно проще, если сначала расставить скобки в соответствии с приоритетом (см. раздел 4.1.2, стр. 190) операторов:

```
++((ret.first)->second); // эквивалентное выражение
```

Рассмотрим это выражение поэтапно.

- `ret` — пара, содержащая значение, возвращаемое функцией `insert()`.
- `ret.first` — первая переменная-член пары, на которую указывает итератор карты, с данным ключом.
- `ret.first->` — обращение к значению итератора, позволяющее получить этот элемент. Элементы карты также являются парами.
- `ret.first->second` — та часть пары элемента карты, которая является значением.
- `++ret.first->second` — инкремент этого значения.

Таким образом, оператор инкремента получает итератор для элемента с ключом слова и увеличивает счетчик, связанный с ключом, для которого не удалась попытка вставки.

Для читателей, использующих устаревший компилятор или код, предшествующий новому стандарту, объявление и инициализация пары `ret` также не совсем очевидны:

```
pair<map<string, size_t>::iterator, bool> ret =  
    word_count.insert(make_pair(word, 1));
```

Здесь определяется пара, вторая переменная-член которой имеет тип `bool`. Понять тип первой переменной-члена этой пары немного труднее. Это тип итератора, определенный типом `map<string, size_t>`.

Добавление элементов в контейнеры `multiset` и `multimap`

Работа программы подсчета слов зависит от того факта, что каждый ключ может присутствовать только однажды. Таким образом, с любым словом будет связан только один счетчик. Но иногда необходима возможность добавить дополнительные элементы с тем же ключом. Например, могло бы понадобиться сопоставить авторов с названиями написанных ими книг. В данном случае для

каждого автора могло бы быть несколько записей, поэтому будет использован контейнер `multimap`, а не `map`. Поскольку ключи контейнеров `multi` не должны быть уникальным, функция `insert()` для них всегда вставляет элемент:

```
multimap<string, string> authors;  
// добавляет первый элемент с ключом Barth, John  
authors.insert({"Barth, John", "Sot-Weed Factor"});  
// ok: добавляет второй элемент с ключом Barth, John  
authors.insert({"Barth, John", "Lost in the Funhouse"});
```

У контейнеров, допускающих совпадение ключей, функция `insert()` получает один элемент и возвращает итератор на новый элемент. Нет никакой необходимости возвращать логическое значение, поскольку в эти контейнеры функция `insert()` всегда добавляет новый элемент.

Упражнения раздела 11.3.2

Упражнение 11.20. Перепишите программу подсчета слов из раздела 11.1 (стр. 539) так, чтобы использовать функцию `insert()` вместо индексации. Какая версия программы по-вашему проще? Объясните почему.

Упражнение 11.21. С учетом того, что `word_count` является картой типов `string` и `size_t`, а также того, что `word` имеет тип `string`, объясните следующий цикл:

```
while (cin >> word)  
    ++word_count.insert({word, 0}).first->second;
```

Упражнение 11.22. С учетом, что `map<string, vector<int>>`, напишите типы, используемые как аргументы, и возвращаемое значение версии функции `insert()`, вставляющей один элемент.

Упражнение 11.23. Перепишите карту, хранящую вектора имен детей с ключом в виде фамилии семьи из упражнений раздела 11.2.1 (стр. 543), так, чтобы использовался контейнер `multimap`.

11.3.3. Удаление элементов

Ассоциативные контейнеры определяют три версии функции `erase()`, описанные в табл. 11.5. Подобно последовательным контейнерам, можно удалить один элемент или диапазон элементов, передав функции `erase()` итератор или пару итераторов. Эти версии функции `erase()` подобны соответствующим функциям последовательных контейнеров: указанный элемент (элементы) удаляется и возвращается тип `void`.

Таблица 11.5. Удаление элементов ассоциативного контейнера

<code>c.erase(k)</code>	Удаляет из карты <code>c</code> элемент с ключом <code>k</code> . Возвращает значение типа <code>size_type</code> , указывающее количество удаленных элементов
<code>c.erase(p)</code>	Удаляет из карты <code>c</code> элемент, обозначенный итератором <code>p</code> . Итератор <code>p</code> должен относиться к фактически существующему элементу карты <code>c</code> , он не может быть равен итератору, возвращаемому функцией <code>c.end()</code> . Возвращает итератор на элемент после позиции <code>p</code> или <code>c.end()</code> , если итератор <code>p</code> обозначает последний элемент контейнера <code>c</code>
<code>c.erase(b, e)</code>	Удаляет элементы в диапазоне, обозначенном парой итераторов <code>b</code> и <code>e</code> . Возвращает итератор <code>e</code>

Ассоциативные контейнеры предоставляют дополнительную версию функции `erase()`, получающую аргумент типа `key_type`. Эта версия удаляет все элементы, если таковые вообще имеются, с заданным ключом и возвращает количество удаленных элементов. Эту версию можно использовать для удаления определенных слов из контейнера `word_count` прежде, чем вывести результат:

```
// удалить по ключу, вернуть количество удаленных элементов
if (word_count.erase(removal_word))
    cout << "ok: " << removal_word << " removed\n";
else cout << "oops: " << removal_word << " not found!\n";
```

Для контейнеров с уникальными ключами функция `erase()` всегда возвращает нуль или единицу. Если возвращается значение нуль, значит, удаляемого элемента не было в контейнере.

Для контейнеров с не уникальными ключами функция `erase()` возвращает количество удаленных элементов и может быть больше единицы:

```
auto cnt = authors.erase("Barth, John");
```

Если `authors` — это контейнер `multimap`, созданный в разделе 11.3.2 (стр. 554), то переменная `cnt` будет содержать значение 2.

11.3.4. Индексация карт

Контейнеры `map` и `unordered_map` предоставляют оператор индексирования и соответствующую функцию `at()` (см. раздел 9.3.2, стр. 447), представленные в табл. 11.6. Типы контейнеров `set` не поддерживают индексацию, поскольку в наборе нет никакого “значения”, связанного с ключом. Элементы сами являются ключами, поэтому операция “доступа к значению, связанному с ключом”, бессмысленна. Нельзя индексировать контейнер `multimap` или `unordered_multimap`, поскольку с заданным ключом может быть ассоциировано несколько значений.

Таблица 11.6. Операторы индексирования контейнеров `map` и `unordered_map`

<code>c[k]</code>	Возвращает элемент с ключом <code>k</code> ; если ключа <code>k</code> нет в контейнере <code>c</code> , добавляется новый элемент, инициализированный значением с ключом <code>k</code>
<code>c.at(k)</code>	Проверяет наличие элемента с ключом <code>k</code> ; если его нет в контейнере <code>c</code> , передает исключение <code>out_of_range</code> (см. раздел 5.6, стр. 257)

Подобно другим использованным ранее операторам индексирования, оператор индексирования карт получает индекс (т.е. ключ) и возвращает связанное с ним значение. Однако, в отличие от других операторов индексирования, если такого ключа еще нет, *создается новый элемент и добавляется* в карту для того ключа. Ассоциированное значение инициализируется значением по умолчанию (см. раздел 3.3.1, стр. 143).

Рассмотрим следующий код:

```
map<string, size_t> word_count; // пустая карта
// вставить инициализированный значением по умолчанию элемент
// с ключом Anna; а затем установить для него значение 1
word_count["Anna"] = 1;
```

Ниже приведена имеющая место последовательность действий.

- В контейнере `word_count` происходит поиск элемента с ключом `Anna`. Элемент не найден.
- В контейнер `word_count` добавляется новая пара ключ–значение. Ключ (константная строка) содержит текст `Anna`. Значение инициализируется по умолчанию, в данном случае нулем.
- Вновь созданному элементу присваивается значение 1.

Поскольку оператор индексирования способен вставить элемент, его можно использовать только для карты, которая не является константной.



Индексация карт существенно отличается от индексации массивов или векторов: использование отсутствующего ключа приводит к *добавлению* элемента с таким ключом в карту.

Использование значения, возвращенного оператором индексирования

Иной способ индексирования карт, отличающий его от других использованных ранее операторов индексирования, влияет на тип возвращаемого значения. Обычно тип, возвращенный в результате обращения к значению итератора, и тип, возвращенный оператором индексирования, совпадают. У карт все не так: при индексировании возвращается объект типа `mapped_type`,

а при обращении к значению итератора карты — объект типа `value_type` (см. раздел 11.3, стр. 548).

Общим у всех операторов индексирования является то, что они возвращают l-значение (см. раздел 4.1.1, стр. 189). Поскольку возвращается l-значение, возможно чтение и запись в элемент:

```
cout << word_count["Anna"]; // получить элемент по индексу Anna;
                               // выводит 1
++word_count["Anna"];       // получить элемент и добавить к нему 1
cout << word_count["Anna"]; // получить элемент и вывести его;
                               // выводит 2
```



В отличие от вектора или строки, тип данных, возвращаемых оператором индексирования карты, отличается из типа, полученного при обращении к значению итератора карты.

Тот факт, что индексирование добавляет элемент, если карта его еще не содержит, позволяет создавать удивительно сжатый код, такой как цикл в программе подсчета слов (см. раздел 11.1, стр. 539). С другой стороны, иногда необходимо только узнать, присутствует ли элемент, но *не добавлять* его в случае отсутствия. В таких случаях не следует использовать оператор индексирования.

Упражнения раздела 11.3.4

Упражнение 11.24. Что делает следующая программа?

```
map<int, int> m;
m[0] = 1;
```

Упражнение 11.25. Сравните следующую программу с предыдущей:

```
vector<int> v;
v[0] = 1;
```

Упражнение 11.26. Какой тип применяется при индексировании карты? Какой тип возвращает оператор индексирования? Приведите конкретный пример, т.е. создайте карту, используйте типы, которые применимы для ее индексирования, а затем выявите типы, которые будет возвращать оператор индексирования.

11.3.5. Доступ к элементам

Ассоциативные контейнеры предоставляют различные способы поиска заданных элементов, описанные в табл. 11.7 (стр. 558). Используемый способ зависит от решаемой задачи. Если нужно лишь выяснить, находится ли некий элемент в контейнере, то, вероятно, лучше использовать функцию `find()`. Для контейнеров, способных содержать только уникальные ключи, вероятно,

не имеет значения, используется ли функция `find()` или `count()`. Но для контейнеров с не уникальными ключами функция `count()` выполняет больше работы: если элемент присутствует, ей все еще нужно подсчитать количество элементов с тем же ключом. Если знать количество не обязательно, лучше использовать функцию `find()`:

```
set<int> iset = {0,1,2,3,4,5,6,7,8,9};
iset.find(1);    // возвращает итератор на элемент с ключом == 1
iset.find(11);   // возвращает итератор == iset.end()
iset.count(1);   // возвращает 1
iset.count(11);  // возвращает 0
```

**Таблица 11.7. Функции поиска элементов
в ассоциативном контейнере**

Функции `lower_bound()` и `upper_bound()` неприменимы для неупорядоченных контейнеров. Оператор индексирования и функция `at()` применимы только для тех контейнеров `map` и `unordered_map`, которые не являются константами.

<code>c.find(k)</code>	Возвращает итератор на (первый) элемент с ключом <code>k</code> или итератор после конца, если такого элемента нет в контейнере
<code>c.count(k)</code>	Возвращает количество элементов с ключом <code>k</code> . Для контейнеров с уникальными ключами результат всегда нуль или единица
<code>c.lower_bound(k)</code>	Возвращает итератор на первый элемент, значение ключа которого не меньше, чем <code>k</code>
<code>c.upper_bound(k)</code>	Возвращает итератор на первый элемент, значение ключа которого больше, чем <code>k</code>
<code>c.equal_range(k)</code>	Возвращает пару итераторов, обозначающих элементы с ключом <code>k</code> . Если такового элемента нет, значение обеих переменных-членов равно <code>c.end()</code>

Использование функции `find()` вместо индексирования карт

Для контейнеров `map` и `unordered_map` оператор индексирования представляет простейший способ поиска значения. Но, как уже упоминалось, у оператора индексирования есть серьезный побочный эффект: если искомого ключа еще нет в карте, индексирование добавляет элемент с таким ключом. Насколько правильно такое поведение, зависит от обстоятельств. Программа подсчета слов полагалась на тот факт, что использование несуществующего ключа при индексировании приводило к вставке элемента с этим ключом и значением 0.

Иногда мы хотим знать, присутствует ли элемент с заданным ключом, не изменяя карту. Нельзя использовать оператор индексирования для определения наличия элемента, поскольку при его отсутствии оператор индексирова-

ния добавит новый элемент с таким ключом. В таких случаях следует использовать функцию `find()`:

```
if (word_count.find("foobar") == word_count.end())
    cout << "foobar is not in the map" << endl;
```

Поиск элементов в контейнерах `multimap` и `multiset`

Поиск элемента в ассоциативном контейнере с уникальными ключами довольно прост — элемент либо есть в контейнере, либо нет. Для контейнеров с не уникальными ключами все несколько сложнее, так как может существовать несколько элементов с заданным ключом. Когда в контейнере `multimap` или `multiset` содержится несколько элементов с одинаковым ключом, они располагаются в контейнере рядом.

Предположим, например, что, имея карту авторов и их книг, следует вывести все книги некоего автора. Эту задачу можно решить тремя способами. Самый очевидный из них — использовать функции `find()` и `count()`:

```
string search_item("Alain de Botton"); // искомый автор
auto entries = authors.count(search_item); // количество записей
auto iter = authors.find(search_item); // первая запись для этого автора
// перебор записей данного автора
while(entries) {
    cout << iter->second << endl; // вывод каждого заглавия
    ++iter; // переход к следующему заглавию
    --entries; // отследить количество выведенных записей
}
```

Код начинается с вызова функции `count()`, позволяющего выяснить количество записей для данного автора, и вызова функции `find()`, позволяющего получить итератор на первый элемент с этим ключом. Количество итераций цикла `for` зависит от числа, возвращенного функцией `count()`. В частности, если функция `count()` возвратит нуль, то цикл не выполнится вообще.



Гарантируется, что перебор контейнера `multimap` или `multiset` возвратит все элементы с заданным ключом.

Другое решение на основании итератора

Задачу можно решить иначе, используя функции `lower_bound()` и `upper_bound()`. Каждая из них получает ключ и возвращает итератор. Если ключ найден в контейнере, функция `lower_bound()` возвратит итератор на первый экземпляр элемента с этим ключом, а итератор, возвращенный функцией `upper_bound()`, указывает на следующий элемент после последнего экземпляра с заданным ключом. Если таковой элемент в контейнере `multimap` отсутствует, то функции `lower_bound()` и `upper_bound()` возвратят одинаковые итераторы на позицию, в которой мог бы находиться такой ключ согласно принятому

порядку. Таким образом, вызов функций `lower_bound()` и `upper_bound()` для того же ключа возвращает диапазон итераторов (см. раздел 9.2.1, стр. 426), обозначающий все элементы с тем же ключом.

Безусловно, возвращенный этими функциями итератор может указывать на элемент непосредственно после конца контейнера. Если искомый элемент имеет самый большой ключ в контейнере, вызов функции `upper_bound()` возвратит итератор на элемент после последнего элемента контейнера. Если элемент отсутствует и ключ является самым большим в контейнере, то вызов функции `lower_bound()` также возвратит итератор на элемент после последнего элемента контейнера.



Итератор, возвращенный функцией `lower_bound()`, может указывать, а может и не указывать на элемент с заданным ключом. Если такового элемента в контейнере нет, функция `lower_bound()` возвращает итератор на первую позицию, в которую, согласно порядку расположения элементов, мог бы быть вставлен элемент с данным ключом.

Используя эти функции, можно переписать программу следующим образом:

```
// определения authors и search_item как прежде
// итераторы beg и end обозначают диапазон элементов данного автора
for (auto beg = authors.lower_bound(search_item),
      end = authors.upper_bound(search_item);
      beg != end; ++beg)
    cout << beg->second << endl; // вывод каждого заглавия
```

Эта программа делает то же, что и предыдущая, использовавшая функции `count()` и `find()`, но более непосредственно. Вызов функции `lower_bound()` устанавливает итератор `beg` так, чтобы он указывал на первый элемент, соответствующий `search_item`, если он есть. Если его нет, то итератор `beg` укажет на первый элемент с ключом, большим, чем `search_item`, который может оказаться итератором после конца. Вызов функции `upper_bound()` присвоит итератору `end` позицию элемента непосредственно после последнего элемента с заданным ключом. Эти функции ничего не говорят о том, присутствует ли данный ключ в контейнере. Важный момент заключается в том, что возвращаемые значения формируют диапазон итераторов (см. раздел 9.2.1, стр. 426).

Если элемента с искомым ключом нет, то возвращаемые функциями `lower_bound()` и `upper_bound()` значения будут равны. Оба, по сути, укажут позицию вставки элемента с указанным ключом при сохранении текущего порядка элементов контейнера.

Если элементы с заданным ключом есть, то итератор `beg` укажет на первый такой элемент. Приращение итератора `beg` позволит перебрать элементы с этим ключом. Равенство итератора `beg` итератору `end` свидетельствует о завершении перебора всех элементов с этим ключом.

Поскольку эти итераторы формируют диапазон, для его перебора можно использовать цикл `for`. Цикл выполняется нуль или большее количество раз, выводя записи для данного автора, если таковые вообще имеются. Если таких элементов нет, то итераторы `beg` и `end` равны и цикл не выполняется ни разу. В противном случае инкремент итератора `beg` в процессе вывода каждой связанной с данным автором записи сравнивает его в конечном счете с итератором `end`.



Если функции `lower_bound()` и `upper_bound()` возвращают тот же итератор, то заданного ключа в контейнере нет.

Функция `equal_range()`

Последний способ решения этой задачи самый простой из всех: вместо функций `upper_bound()` и `lower_bound()` можно вызвать функцию `equal_range()`.

Эта функция получает ключ и возвращает пару итераторов. Если элементы с таким ключом в контейнере присутствуют, то первый итератор укажет на первый экземпляр элемента, а второй — на следующий после последнего экземпляра. Если подходящего элемента нет, то первый и второй итераторы укажут позицию, в которую этот элемент может быть вставлен.

Функцию `equal_range()` можно использовать для еще одного изменения программы:

```
// определения authors и search_item, как прежде
// pos содержит итераторы, обозначающие диапазон элементов
// с заданным ключом
for (auto pos = authors.equal_range(search_item);
     pos.first != pos.second; ++pos.first)
    cout << pos.first->second << endl; // вывод каждого заглавия
```

Эта программа очень похожа на предыдущую, где использовались функции `upper_bound()` и `lower_bound()`. Для хранения диапазона итераторов вместо локальных переменных `beg` и `end` используется пара, возвращенная функцией `equal_range()`. Переменная-член `first` этой пары содержит тот же итератор, который возвратила бы функция `lower_bound()`, а переменная-член `second` — итератор, который возвратила бы функция `upper_bound()`. Таким образом, в этой программе значение `pos.first` эквивалентно значению `beg`, а `pos.second` — значению `end`.

Упражнения раздела 11.3.5

Упражнение 11.27. Для решения каких видов задач используется функция `count()`? Когда вместо нее можно использовать функцию `find()`?

Упражнение 11.28. Определите и инициализируйте переменную, содержащую результат вызова функции `find()` для карты строк и векторов целых чисел.

Упражнение 11.29. Что возвращают функции `upper_bound()`, `lower_bound()` и `equal_range()`, когда им передается ключ, отсутствующий в контейнере?

Упражнение 11.30. Объясните значение операнда `pos.first->second`, использованного в выражении вывода последней программы данного раздела.

Упражнение 11.31. Напишите программу, определяющую контейнер `multimap` авторов и их работ. Используйте функцию `find()` для поиска элемента и его удаления. Убедитесь в корректности работы программы, когда искомого элемента нет в карте.

Упражнение 11.32. Используя контейнер `multimap` из предыдущего упражнения, напишите программу вывода списка авторов и их работ в алфавитном порядке.

11.3.6. Карта преобразования слов

Завершим этот раздел программой, иллюстрирующей создание, поиск и перебор карты. Программа получает одну строку и преобразует ее в другую. Программе передаются два файла: первый содержит правила, используемые для преобразования текста во втором файле. Каждое правило состоит из слова, которое может встретиться во входном файле, и фразы, используемой вместо него. Идея в том, чтобы, встретив во вводе некое слово, заменить его соответствующей фразой. Второй файл содержит преобразуемый текст.

Вот содержимое файла преобразования слов.

```
brb be right back
k okay?
y why
r are
u you
pic picture
thk thanks!
l8r later
```

Подлежащий преобразованию текст таков:

```
where r u
y dont u send me a pic
k thk l8r
```

Программа должна создать следующий вывод:

```
where are you
why dont you send me a picture
okay? thanks! later
```

Программа преобразования слова

Решение подразумевает использование трех функций. Функция `word_transform()` будет осуществлять общую обработку. Потребуется два аргумента типа `ifstream`: первый будет связан с файлом преобразования слов, а второй — с текстовым файлом, который предстоит преобразовать. Функция `buildMap()` будет читать файл правил преобразования и создавать элемент карты для каждого слова и результата его преобразования. Функция `transform()` получит строку и, если она есть в карте, возвратит результат преобразования.

Давайте начнем с определения функции `word_transform()`. Важнейшие ее части — вызовы функций `buildMap()` и `transform()`:

```
void word_transform(ifstream &map_file, ifstream &input)
{
    auto trans_map = buildMap(map_file); // хранит преобразования
    string text; // содержит каждую строку из ввода
    while (getline(input, text)) { // читать строку из ввода
        istringstream stream(text); // читать каждое слово
        string word;
        bool firstword = true; // контролирует вывод пробела
        while (stream >> word) {
            if (firstword)
                firstword = false;
            else
                cout << " "; // вывод пробела между словами
            // transform() возвращает свой первый аргумент или
            // результат преобразования
            cout << transform(word, trans_map); // вывод результата
        }
        cout << endl; // обработка текущей строки ввода окончена
    }
}
```

Функция начинается вызовом функции `buildMap()`, создающим карту преобразования слов. Результат сохраняется в карте `trans_map`. Остальная часть функции обрабатывает входной файл. Цикл `while` использует функцию `getline()` для чтения входного файла по одной строке за раз. Построчно чтение осуществляется для того, чтобы строки вывода заканчивались там же, где и строки входного файла. Для получения слов каждой строки используется вложенный цикл `while`, использующий строковый поток `istringstream` (см. раздел 8.3, стр. 413) для обработки каждого слова текущей строки.

Внутренний цикл `while` выводит результат, используя логическую переменную `firstword`, чтобы решить, выводить ли пробел. Вызов функции `transform()` получает подлежащее выводу слово. Значение, возвращенное функцией `transform()`, будет либо исходным словом строки, либо соответствующим ему преобразованием из карты `trans_map`.

Создание карты преобразования

Функция `buildMap()` читает переданный ей файл и создает карту преобразований.

```
map<string, string> buildMap(istream &map_file)
{
    map<string, string> trans_map; // хранит преобразования
    string key; // слово для преобразования
    string value; // фраза, используемая вместо него
    // прочитать первое слово в ключ, а остальную часть строки в значение
    while (map_file >> key && getline(map_file, value))
        if (value.size() > 1) // проверить, есть ли преобразование
            trans_map[key] = value.substr(1); // убрать предваряющий
            // пробел
        else
            throw runtime_error("no rule for " + key);
    return trans_map;
}
```

Каждая строка файла `map_file` соответствует правилу. Каждое правило — это слово, сопровождаемое фразой, способной содержать несколько слов. Для чтения слов, преобразуемых в ключи, используется оператор `>>` и функция `getline()` для чтения остальной части строки в значение. Поскольку функция `getline()` не отбрасывает предваряющие пробелы (см. раздел 3.2.2, стр. 130), необходимо убрать пробел между словом и соответствующим ему правилом. Прежде чем сохранить преобразование, осуществляется проверка наличия в нем хотя бы одного символа. Если это так, то происходит вызов функции `substr()` (см. раздел 9.5.1, стр. 463), позволяющий устранить пробел, отделяющий фразу преобразования от соответствующего ему слова, и сохранить эту подстроку в карте `trans_map`.

Обратите внимание на использование оператора индексирования при добавлении пары ключ–значение. При этом неявно игнорируется происходящее при повторении слова в файле преобразования. Если слово повторяется несколько раз, то в карте `trans_map` окажется последняя соответствующая фраза. По завершении цикла `while` карта `trans_map` содержит все данные, необходимые для преобразования ввода.

Осуществление преобразования

Фактическое преобразование осуществляет функция `transform()`. Ее параметры — ссылки на преобразуемую строку и карту преобразования. Если переданная строка находится в карте, функция `transform()` возвращает соответствующую ей фразу преобразования. Если переданной строки в карте нет, функция `transform()` возвращает свой аргумент:

```
const string &
transform(const string &s, const map<string, string> &m)
{
    // фактическая работа карты; это основная часть программы
    auto map_it = m.find(s);
    // если слово есть в карте преобразования
    if (map_it != m.cend())
        return map_it->second; // использовать замену слова
    else
        return s; // в противном случае вернуть исходное слово
}
```

Код начинается с вызова функции `find()`, позволяющего определить, находится ли данная строка в карте. Если это так, то функция `find()` возвращает итератор на соответствующий элемент. В противном случае функция `find()` возвращает итератор на элемент после конца. Если элемент найден, обращение к значению итератора возвращает пару, содержащую ключ и значение этого элемента (см. раздел 11.3, стр. 548). Функция возвращает значение переменной-члена `second` этой пары, являющееся преобразованной фразой, используемой вместо строки `s`.

Упражнения раздела 11.3.6

Упражнение 11.33. Реализуйте собственную версию программы преобразования слов.

Упражнение 11.34. Что будет, если в функции `transform()` вместо функции `find()` использовать оператор индексирования?

Упражнение 11.35. Что будет (если будет) при таком изменении функции `buildMap()`:

```
trans_map[key] = value.substr(1);
as trans_map.insert({key, value.substr(1)})?
```

Упражнение 11.36. Текущая версия программы не проверяет допустимость входного файла. В частности, она подразумевает, что все правила в файле преобразований корректны. Что будет, если строка в этом файле содержит ключ, один пробел и больше ничего? Проверьте свой ответ на текущей версии программы.



11.4. Неупорядоченные контейнеры



Новый стандарт определяет четыре *неупорядоченных ассоциативных контейнера* (*unordered container*). Вместо оператора сравнения для организации своих элементов эти контейнеры используют *хеш-функцию* (*hash function*) и оператор `==` типа ключа. Неупорядоченный контейнер особенно полезен, когда имеющийся тип ключа не дает очевидных отношений для упорядочивания элементов. Эти контейнеры полезны также в приложениях, где цена упорядочивания элементов высока.

Хотя в принципе хеширование обеспечивает лучшую среднюю производительность, достижение хороших результатов на практике зачастую требует серьезной проверки производительности и настройки. В результате обычно проще (а зачастую и производительней) использовать упорядоченный контейнер.



Используйте неупорядоченный контейнер, если тип ключа принципиально неупорядочен или если проверка производительности свидетельствует о проблеме, решить которую позволит только хеширование.

Использование неупорядоченного контейнера

Кроме функций управления хешированием, неупорядоченные контейнеры предоставляют те же функции (`find()`, `insert()` и т.д.), что и упорядоченные контейнеры. Это значит, что функции, использовавшиеся для контейнеров `map` и `set`, применимы также к контейнерам `unordered_map` и `unordered_set`. Аналогично неупорядоченные контейнеры имеют версии с не уникальными ключами.

В результате вместо соответствующего упорядоченного контейнера можно обычно использовать неупорядоченный контейнер, и наоборот. Но поскольку элементы хранятся неупорядочено, вывод программы, использующей неупорядоченный контейнер, будет (обычно) отличаться от такового при использовании упорядоченного контейнера.

Например, первоначальную программу подсчета слов из раздела 11.1 (стр. 539) можно переписать так, чтобы использовать контейнер `unordered_map`:

```
// подсчет слов, но слова не в алфавитном порядке
unordered_map<string, size_t> word_count;
string word;
while (cin >> word)
    ++word_count[word]; // получить и прирастить счетчик слов
for (const auto &w : word_count) // для каждого элемента карты
    // отобразить результаты
    cout << w.first << " occurs " << w.second
        << ((w.second > 1) ? " times" : " time") << endl;
```

Единственное различие между этой программой и первоначальной заключается в типе `word_count`. Если запустить эту версию для того же ввода, то получится то же количество для каждого слова.

```
containers. occurs 1 time
use occurs 1 time
can occurs 1 time
examples occurs 1 time
...
```

Но вывод вряд ли будет в алфавитном порядке.

Управление ячейками

Неупорядоченные контейнеры организованы как коллекция ячеек, каждая из которых содержит любое количество элементов. Для группировки элементов в ячейки контейнер использует хеш-функцию. Для доступа к элементу контейнер сначала вычисляет хеш-код элемента, указывающий, где искать ячейку. Контейнер помещает все элементы с одинаковым значением хеш-функции в ту же ячейку. Если контейнер допускает несколько элементов с одинаковым ключом, то все элементы с этим ключом будут находиться в той же ячейке. В результате производительность неупорядоченного контейнера зависит от качества его хеш-функции, количества и размера его ячеек.

Хеш-функция всегда возвращает тот же результат, будучи вызванной с тем же аргументом. В идеале хеш-функция сопоставляет также каждое конкретное значение с уникальной ячейкой. Однако хеш-функция может сопоставить элементы с разными ключами с той же ячейкой. Когда ячейка содержит несколько элементов, поиск искомого среди них осуществляется последовательно. Как правило, вычисление хеш-кода элемента и поиск его ячейки осуществляется очень быстро. Но если в ячейке много элементов, то для поиска определенного элемента может потребоваться много сравнений.

Неупорядоченные контейнеры предоставляют набор перечисленных в табл. 11.8 функций, позволяющих управлять ячейками. Эти функции-члены позволяют запрашивать состояние контейнера и реорганизовать его по мере необходимости.

Таблица 11.8. Функции управления неупорядоченным контейнером

Взаимодействие с ячейками	
<code>c.bucket_count()</code>	Количество используемых ячеек
<code>c.max_bucket_count()</code>	Наибольшее количество ячеек, которое может содержать данный контейнер
<code>c.bucket_size(n)</code>	Количество элементов в ячейке <code>n</code>
<code>c.bucket(k)</code>	Ячейка, в которой следует искать элементы с ключом <code>k</code>

Перебор ячеек	
<code>local_iterator</code>	Тип итератора, способный обращаться к элементам в ячейке
<code>const_local_iterator</code>	Константная версия итератора ячейки
<code>c.begin(n), c.end(n)</code>	Итераторы на первый и следующий после последнего элементы ячейки <code>n</code>
<code>c.cbegin(n), c.cend(n)</code>	Возвращают итератор <code>const_local_iterator</code>
Политика хеша	
<code>c.load_factor()</code>	Среднее количество элементов на ячейку. Возвращает тип <code>float</code>
<code>c.max_load_factor()</code>	Средний размер ячейки, который пытается поддерживать контейнер <code>c</code> . Контейнер <code>c</code> добавляет ячейки, чтобы сохранить соотношение <code>load_factor <= max_load_factor</code> . Возвращает тип <code>float</code>
<code>c.rehash(n)</code>	Реорганизует хранилище так, чтобы <code>bucket_count >= n</code> и <code>bucket_count > size/max_load_factor</code>
<code>c.reserve(n)</code>	Реорганизует контейнер <code>c</code> так, чтобы он мог содержать <code>n</code> элементов без вызова функции <code>rehash()</code>

Требования к типу ключа неупорядоченных контейнеров

По умолчанию для сравнения элементов неупорядоченные контейнеры используют оператор `==` типа ключа. Они также используют объект типа `hash<key_type>` при создании хеш-кода для каждого элемента. Библиотека предоставляет также версии шаблона *хеша* для встроенных типов, включая указатели. Она определяет также шаблон `hash` для некоторых из библиотечных типов, включая строки и интеллектуальные указатели, которые рассматривались в главе 12. Таким образом, можно непосредственно создать неупорядоченный контейнер, ключ которого имеет один из встроенных типов (включающий типы указателей) либо тип `string` или интеллектуального указателя.

Однако нельзя непосредственно определить неупорядоченный контейнер, использующий для ключа собственные типы классов. В отличие от контейнеров, шаблон хеша нельзя использовать непосредственно. Вместо этого придется предоставить собственную версию шаблона `hash`. Это будет описано в разделе 16.5 (стр. 890).

Вместо хеша по умолчанию можно применить стратегию, подобную используемой при переопределении заданного по умолчанию оператора сравнения ключей упорядоченных контейнеров (см. раздел 11.2.2, стр. 543). Чтобы

использовать тип `Sales_data` для ключа, необходимо предоставить функцию для замены оператора `==` и вычисления хеш-кода. Начнем с определения этих функций:

```
size_t hasher(const Sales_data &sd)
{
    return hash<string>()(sd.isbn());
}
bool eqOp(const Sales_data &lhs, const Sales_data &rhs)
{
    return lhs.isbn() == rhs.isbn();
}
```

Чтобы создать хеш-код для переменной-члена ISBN, функция `hasher()` использует объект библиотечного типа `hash` для типа `string`. Точно так же функция `eqOp()` сравнивает два объекта класса `Sales_data`, сравнивая их ISBN.

Эти функции можно также использовать для определения контейнера `unordered_multiset` следующим образом:

```
using SD_multiset = unordered_multiset<Sales_data,
                                     decltype(hasher)*, decltype(eqOp)*>;
// аргументы - размер ячейки, указатель на оператор равенства и
// хеш-функцию
SD_multiset bookstore(42, hasher, eqOp);
```

Чтобы упростить объявление `bookstore`, определим сначала псевдоним типа (см. раздел 2.5.1, стр. 106) для контейнера `unordered_multiset`, у хеша и оператора равенства которого есть те же типы, что и у функций `hasher()` и `eqOp()`. Используя этот тип, определим `bookstore`, передав указатели на функции, которые он должен использовать.

Если у класса есть собственный оператор `==`, можно переопределить только хеш-функцию:

```
// использовать FooHash для создания хеш-кода;
// у Foo должен быть оператор ==
unordered_set<Foo, decltype(FooHash)*> fooSet(10, FooHash);
```

Упражнения раздела 11.4

Упражнение 11.37. Каковы преимущества неупорядоченного контейнера по сравнению с упорядоченной версией этого контейнера? Каковы преимущества упорядоченной версии?

Упражнение 11.38. Перепишите программы подсчета слов (см. раздел 11.1, стр. 539) и преобразования слов (см. раздел 11.3.6, стр. 562) так, чтобы использовать контейнер `unordered_map`.

Резюме

Ассоциативные контейнеры обеспечивают эффективный поиск и возвращение элементов по ключу. Использование ключа отличает ассоциативные контейнеры от последовательных, в которых к элементам обращаются по позиции.

Существует восемь ассоциативных контейнеров со следующими свойствами.

- Карта хранит пары ключ–значение; набор хранит только ключи.
- Есть контейнеры с уникальными ключами и с не уникальными.
- Ключи могут храниться упорядоченными или нет.

Упорядоченные контейнеры используют функцию сравнения для упорядочивания элементов по ключу. По умолчанию для сравнения используется оператор `<` типа ключа. Неупорядоченные контейнеры используют для организации своих элементов оператор `==` типа ключа и объект типа `hash<key_type>`.

Имена контейнеров с не уникальными ключами включают слово `multi`; а имена контейнеров, использующих хеширование, начинаются словом `unordered`. Контейнер `set` — это упорядоченная коллекция, каждый ключ которой уникален; контейнер `unordered_multiset` — это неупорядоченная коллекция, ключи которой могут повторяться.

Ассоциативные контейнеры имеют много общих операций с последовательными контейнерами. Но ассоциативные контейнеры определяют некоторые новые функции и переопределяют значение и типы возвращаемого значения некоторых функций, общих для последовательных и ассоциативных контейнеров. Различия в функциях отражают способ использования ключей в ассоциативных контейнерах.

Итераторы упорядоченных контейнеров обеспечивают доступ к элементам по ключу. Элементы с тем же ключом хранятся рядом друг с другом и в упорядоченных, и в неупорядоченных контейнерах.

Термины

Ассоциативный контейнер (associative container). Тип, содержащий коллекцию объектов и обеспечивающий эффективный поиск по ключу.

Ассоциативный массив (associative array). Массив, элементы которого проиндексированы по ключу, а не по позиции. Таким образом, массив сопоставляет (ассоциирует) ключ со значением.

Контейнер map (карта). Ассоциативный контейнер, аналогичный ассоциативному массиву. Подобно типу `vector`, тип `map` является шаблоном класса. Но при создании карты необходимо указать два типа: тип ключа и тип связанного с ним значения. В контейнере `map` ключи уникальны, они не повторяются. Каждый ключ связан с определенным значением. Обращение к значению итератора карты возвращает

объект типа `pair`, который содержит константный ключ и связанное (ассоциированное) с ним значение.

Контейнер `multimap`. Ассоциативный контейнер, подобный контейнеру `map`, но способный содержать одинаковые ключи.

Контейнер `multiset`. Ассоциативный контейнер, который содержит только ключи. В отличие от набора, способен содержать одинаковые ключи.

Контейнер `set` (набор). Ассоциативный контейнер, который содержит только ключи. Ключи в контейнере `set` не могут совпадать.

Контейнер `unordered_map`. Контейнер, элементы которого являются парами ключ–значение. Допустим только один элемент на ключ.

Контейнер `unordered_multimap`. Контейнер, элементы которого являются парами ключ–значение. Допустимо несколько элементов на ключ.

Контейнер `unordered_multiset`. Контейнер, хранящий ключи. Допустимо несколько элементов на ключ.

Контейнер `unordered_set`. Контейнер, хранящий ключи. Допустим только один элемент на ключ.

Неупорядоченный контейнер (`unordered container`). Ассоциативные контейнеры, использующие хеширование, а не сравнение ключей для хранения и доступа к элементам. Эффективность этих контейнеров зависит от качества хеш-функции.

Оператор `*`. Оператор обращения к значению, примененный к итератору контейнера `map`, `set`, `multimap` или `multiset`, возвращает объект типа `value_type`. Обратите внимание на то, что типом `value_type` контейнера `map` и `multimap` является пара (`pair`).

Оператор `[]`. Оператор индексирования, примененный к контейнеру `map`, получает индекс, типом которого должен быть `key_type` (или тип, допускающий преобразование в него). Возвращает значение типа `mapped_type`.

Строгое сравнение (`strict weak ordering`). Отношения между ключами ассоциативного контейнера. При строгом сравнении можно сравнить два любых значения и выяснить, которое из них меньше. Если ни одно из значений не меньше другого, они считаются равными.

Тип `key_type`. Тип, определенный в шаблоне ассоциативного контейнера, которому соответствует тип ключей, используемых для сохранения и возвращения значения. У контейнера `map` тип `key_type` используется для индексации. У контейнера `set` типы `key_type` и `value_type` совпадают.

Тип `mapped_type`. Тип, определенный в шаблонах ассоциативных контейнеров `map` и `multimap`, которому соответствует тип хранимых значений.

Тип `pair` (пара). Тип, объект которого содержит две открытые переменные-члена по имени `first` (первый) и `second` (второй). Тип `pair` является шаблоном, при создании класса которого указывают два типа: тип первого и тип второго элемента.

Тип `value_type`. Тип элемента, хранимого в контейнере. У контейнеров `set` и `multiset` типы `value_type` и `key_type` совпадают. У контейнеров `map` и `multimap` этот тип представляет собой пару, первый элемент которой (`first`) имеет тип `const key_type`, а второй (`second`) — тип `mapped_type`.

Хеш (hash). Специальный библиотечный шаблон, который используют неупорядоченные контейнеры для управления позицией элементов.

Хеш-функция (hash function). Функция, сопоставляющая значения заданного типа с целочисленными значениями (`size_t`). Равные значения должны сопоставляться с равными целыми числами; неравные значения должны сопоставляться с неравными целыми числами, если это возможно.